

```
--- djangoapi\scripts\crop\DjangoModels\parcelas\parcelasDJ.txt ---
```

```
from crop.models import Parcelas
from scripts.crop.DjangoModels.tablasDJ import TablasDJ
from django.contrib.gis.geos import GEOSGeometry

class ParcelasDJ(TablasDJ):
    def __init__(self):
        super().__init__(Parcelas)

    def insert(self, d: dict):
        # 1. Snapping y GEOS
        wkb = self._get_snapped_wkb(d['geom'])
        g = GEOSGeometry(wkb, srid=self.srid)

        if not g.valid:
            return {'ok': False, 'message': 'Geometría inválida'}

        # 2. Validación T*****
        if self._check_interior_intersection(wkb):
            return {'ok': False, 'message': 'Intersección de interiores detectada'}

        # 3. Mapeo de campos y cálculo automático
        d['geom'] = g
        d['area'] = g.area
        d['perimetro'] = g.length

        obj = self.model(**d)
        obj.save()
        return {'ok': True, 'id': obj.id}

    def update(self, d: dict):
        """Actualiza un registro existente."""
        try:
            obj = self.model.objects.get(id=d['id'])
            wkb = self._get_snapped_wkb(d['geom'])
            g = GEOSGeometry(wkb, srid=self.srid)

            if not g.valid: return {'ok': False, 'message': 'Geometría inválida'}

            if self._check_interior_intersection(wkb, exclude_id=d['id']):
                return {'ok': False, 'message': 'La actualización genera una
intersección con otra parcela'}

            # Actualizamos atributos
            obj.dueno = d.get('dueno', obj.dueno)
            obj.cultivo = d.get('cultivo', obj.cultivo)
            obj.geom = g
            obj.area = g.area
            obj.perimetro = g.length
            obj.save()
            return {'ok': True, 'message': 'Parcela actualizada'}
        except self.model.DoesNotExist:
```

```
return {'ok': False, 'message': 'ID no encontrado'}
```